International Institute of Information Technology, Hyderabad
Digital VLSI Design

Assignment: RTL to GDSII

4-bit × 4-bit Array Multiplier on SkyWater SKY130

Manas Inamdar
Roll No: 2023102052

Soham Jahagirdar
Roll No: 2023102046

Contents

1 Introduction and Methodology 1
2 Design and Verification (Part A) 2
3 Logic Synthesis (Part B) 3
4 Static Timing Analysis (Part C) 3
5 Physical Design (Part D) 4
6 Answers to the Questions 7
7 Observations and Conclusions 9

1 Introduction and Methodology

For this assignment we designed a registered 4-bit × 4-bit unsigned array multiplier in Verilog and took it through a complete open-source RTL-to-GDSII flow on the SkyWater SKY130 process (`sky130_fd_sc_hd` standard-cell library). The design was verified exhaustively at every stage, synthesized to standard cells, timed with static timing analysis, and placed and routed to a final layout that passes timing, DRC and LVS signoff.

All tools come from the IIC-OSIC-TOOLS Docker image `hpretl/iic-osic-tools:2026.09`: Icarus Verilog 14.0, Yosys 0.69, OpenSTA 3.1.0, LibreLane 3.1.0.dev3 (with OpenROAD, Magic 8.3, KLayout 0.30.12 and Netgen 1.5). The repository README lists the exact command of every step in order.

1.1 Design Flow

The flow consists of the five stages shown in Figure 1, one for each part of the assignment.

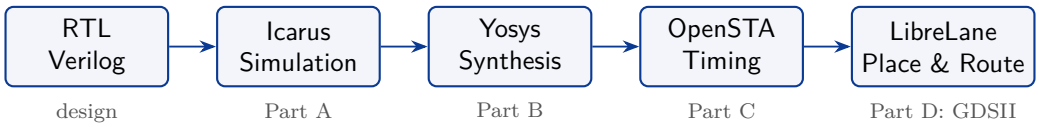


Figure 1: RTL-to-GDSII flow used in this assignment

2 Design and Verification (Part A)

2.1 RTL Design

The module `full_adder` is written with two continuous assignments:

$$sum = a \oplus b \oplus cin \tag{1}$$

$$cout = ab + a\,cin + b\,cin \tag{2}$$

The top module `mult_array` registers the operands `a` and `b` into `a_reg` and `b_reg` on every rising clock edge. From the registered values it forms the 16 partial products

$$pp_i[j] = \texttt{a_reg}[j] \cdot \texttt{b_reg}[i], \qquad i, j \in \{0, 1, 2, 3\} \tag{3}$$

with explicit AND assignments, and adds them with 12 `full_adder` instances arranged as three 4-bit ripple-carry rows, wired exactly as in the connection table of the assignment (Figure 2). The 8-bit result is registered into `p`. All 16 flip-flops are cleared by the asynchronous active-low reset `rst_n`, and the design does not use the `*` operator. Operands captured at edge 1 produce the product at edge 2, which gives a latency of two clock cycles and a throughput of one result per cycle.

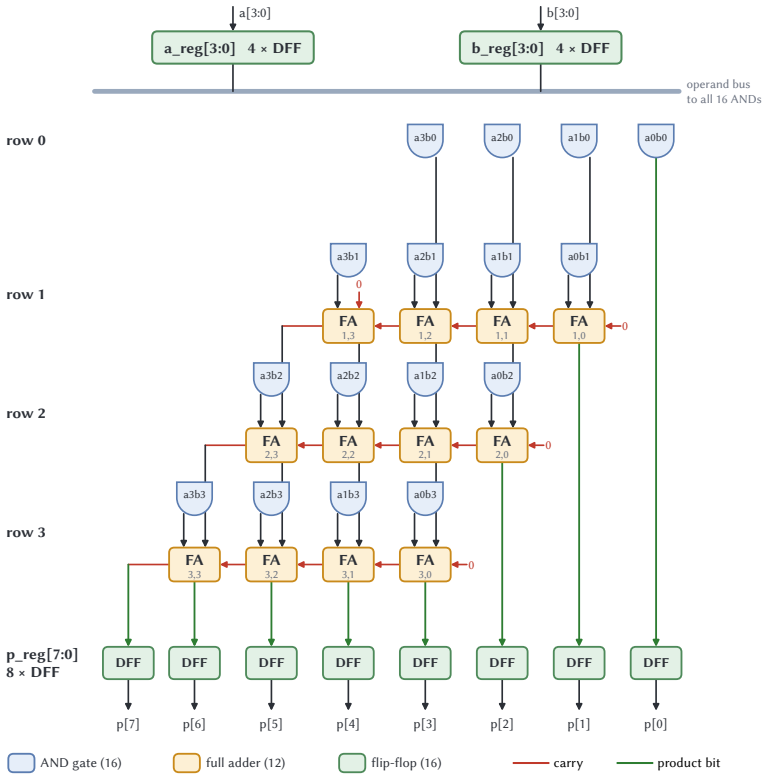


Figure 2: Block diagram of `mult_array`: input registers, 16 AND gates, 12 full adders FA(i,j) in rows 1–3, and the 8-bit output register. Columns correspond to bit weights, so each sum passes straight down to the next row while carries ripple from right to left

2.2 Testbench and Results

The testbench `tb_mult` generates a 10 ns clock and holds `rst_n` low for the first two rising edges. It then applies all 256 combinations of `a` and `b` on falling clock edges, waits for two rising edges and compares `p` with `a*b` (the only place the multiplication operator is used). It prints every mismatch, exactly one summary line, and writes the waveform to `mult.vcd`. For the error test (task 6) we changed the sum equation to `a ^ b` in a copy of `full_adder.v`:

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim
PASS: 256 vectors checked, 0 errors

# error test: sum = a ^ b
MISMATCH: a=3 b=3 expected=9 actual=5            (95 mismatch lines in total)
FAIL: 256 vectors checked, 95 errors
```

Dropping `cin` from the sum corrupts every product that needs a carry into a sum bit, and 95 of the 256 products fail. With the original equation restored the design passes again.

2.3 Hand-worked Example

We used the roll number of Manas Inamdar, 2023102052. Its last digit is 2, so $A = 2 + 5 = 7$, and because $A = 7$ the assignment sets $B = 9$. The left box follows the layout of Section 2 of the assignment; the right box traces the same product through the rows of the array.

```
          0 1 1 1   A = 7
      x   1 0 0 1   B = 9
  ---------------
          0 1 1 1   pp0  (b0 = 1)
        0 0 0 0     pp1  (b1 = 0)
      0 0 0 0       pp2  (b2 = 0)
    0 1 1 1         pp3  (b3 = 1)
  ---------------
  0 0 1 1 1 1 1 1   P = 63
```

```
row 0  pp0 = 0111            p[0]=1  r0 = 0011
row 1  0000 + 0011 = 0011    p[1]=1  r1 = 0001
row 2  0000 + 0001 = 0001    p[2]=1  r2 = 0000
row 3  0111 + 0000 = 0111    p[3]=1
       p[7:4] = {c3[4], s3[3:1]} = 0011

P = 0011 1111 = 63 = 7 x 9
```

2.4 Waveform

Figure 3 shows the same transaction in the RTL simulation. The operands appear on a falling edge, are captured on the next rising edge, and the product is registered one cycle later.

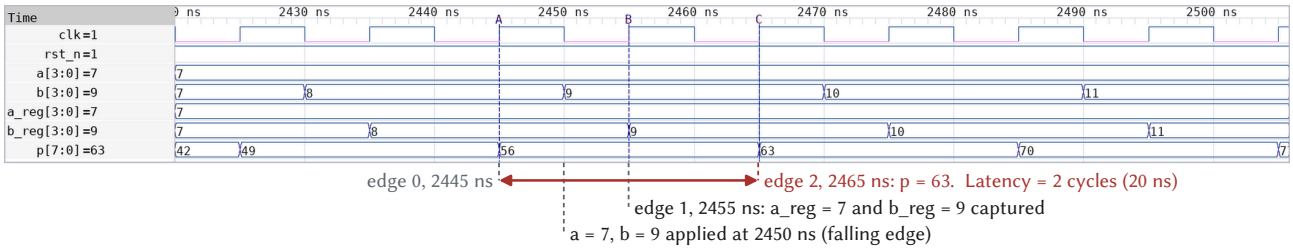


Figure 3: GTKWave view of `mult.vcd` for $A = 7$, $B = 9$ (colours inverted for print). The operands are applied at 2450 ns, captured at edge 1 (2455 ns), and $p = 63$ is registered at edge 2 (2465 ns): a latency of two clock cycles. Markers A, B and C sit on edges 0, 1 and 2

3 Logic Synthesis (Part B)

The script `synth/synth.tcl` performs the seven required Yosys steps with the `tt_025C_1v80` Liberty library: reading the RTL, generic synthesis with flattening (`synth -flatten`), flip-flop mapping (`dfflibmap`), gate mapping (`abc`), removal of unused wires (`opt_clean -purge`), the cell and area report (`stat -liberty`), and the gate-level netlist (`write_verilog`). The mapped netlist was then simulated with the same testbench and the SKY130 functional cell models (`-DFUNCTIONAL`, unit delay); it passes all 256 vectors.

4 Static Timing Analysis (Part C)

The script `sta/sta.tcl` reads the Liberty library and the netlist, links `mult_array`, defines `clk` with period T, applies 1.0 ns input and output delays, prints the worst setup slack and writes the critical-

Metric	Value
Total number of cells	63
Number of flip-flops	16 (`dfrtp_1`)
Total cell area	740.7104 µm^2
Flip-flop area	$16 \times 25.024 = 400.384$ µm^2
Flip-flop area (% of total)	54.05 %
Three most-used cell types	`dfrtp_1` (16), `nand2_1` (8), `a21oi_1` (6)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 1: Synthesis results from `synth/mult_array_stat.rpt`

path report. At $T = 10$ ns the worst slack (WNS) is 7.41 ns, so

$$T_{\min} = T - \text{WNS} = 10 - 7.41 = 2.59 \text{ ns}, \tag{4}$$

$$F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.59} = 386.10 \text{ MHz}. \tag{5}$$

Re-running at $T = 2.59$ ns reports a worst slack of -0.00 ns (0.000 ns in the detailed report, -0.00025 ns at full precision): rounding $T_{\min}$ to 0.01 ns lands just below the limit, and at 2.60 ns the slack is $+0.01$ ns. The slack at $T_{\min}$ is therefore close to zero, as the assignment requires.

Metric	Value
Worst slack at $T = 10$ ns	7.410 ns
$T_{\min}$	2.59 ns
$F_{\max}$	386.10 MHz
Slack at $T = T_{\min}$	0.000 ns (≈ 0)
Critical path start point	`_087_`, input register `a_reg[1]` (`dfrtp_1`)
Critical path end point	`_100_`, output register `p[6]` (`dfrtp_1`)
Number of cells on the critical path	10 (8 logic cells, launch and capture flip-flops)
Latency $= 2 \times T_{\min}$	5.18 ns
Throughput	386.10 million results per second

Table 2: Static timing results at the typical corner (ideal clock, no wire parasitics)

In RTL terms the path runs `a_reg[1]` → FA(1,0) → FA(1,1) → FA(2,0) → FA(2,1) → FA(2,2) → FA(3,1) → FA(3,2) → FA(3,3) = `p[6]`: **eight full-adder stages, matching the** $2N = 8$ **architectural estimate**, first down the rows, then along the carry chain of row 3. Six of the eight logic cells each compute one full-adder output. FA(1,0) has a constant-zero carry input, so synthesis merged it with the partial-product gates and FA(1,1) into two cells, the inverter and the `o311ai`. STA reports the longest *structural* path and does not check whether it can be sensitized: `_047_` only responds to `a_reg[1]` when `a_reg[0]` = 0, which forces `_050_` low, so this exact path is a conservative bound rather than a delay a single input change can produce.

5 Physical Design (Part D)

The configuration `pnr/config.json` sets `DESIGN_NAME`, `VERILOG_FILES` (the RTL), `CLOCK_PORT`, `FP_SIZING = "absolute"`, `DIE_AREA = [0, 0, 150, 150]`, the PDK and cell library, and `CLOCK_PERIOD`. The first period was

$$1.2 \times T_{\min} = 1.2 \times 2.59 = 3.108 \text{ ns} \rightarrow 3.5 \text{ ns (next 0.5 ns)}. \tag{6}$$

Instance	Cell	Arrival (ns)	Function in the RTL
_087_	dfrtp_1	0.398	launch flip-flop, a_reg[1]
_039_	clkinv_1	0.471	$\overline{\texttt{a_reg[1]}}$
_047_	o311ai_0	0.827	partial products merged with FA(1,0), FA(1,1)
_050_	a21oi_1	0.999	carry of FA(2,0), c2[1]
_063_	maj3_1	1.381	carry of FA(2,1), c2[2]
_064_	xnor2_1	1.541	sum of FA(2,2), $\overline{\texttt{s2[2]}}$
_073_	maj3_1	1.924	carry of FA(3,1), $\overline{\texttt{c3[2]}}$
_075_	maj3_1	2.281	carry of FA(3,2), $\overline{\texttt{c3[3]}}$
_079_	xnor3_1	2.457	sum of FA(3,3), s3[3] = p[6]
_100_	dfrtp_1	setup 0.134	capture flip-flop, p[6]

Table 3: Cells on the critical path at $T = 2.59$ ns. The last column was found by simulating the netlist and the RTL for all 256 register values and matching the truth table of each net to the RTL signals

LibreLane signs off setup and hold timing at nine corners (typical, slow and fast process, each with nominal, minimum and maximum parasitics), and also checks maximum slew, capacitance and fanout. The period was raised by 0.5 ns per run until all checks passed (Table 4).

The first series used only the required keys. It closed setup timing at 6.5 ns, but every run left 11 maximum-slew violations on the reset net: the flow drove ten RESET_B pins from one weak clock-delay buffer (clkdlybuf4s50_1), giving a 0.778 ns transition against the 0.75 ns limit at ss_100C_1v60. LibreLane repairs slew before routing and at the typical corner, so it never saw this. The final configuration adds two electrical-repair settings, without changing any timing constraint: RUN_POST_GRT_DESIGN_REPAIR = true repeats the slew, capacitance and fanout repair after global routing with real wire loads, and GRT_DESIGN_REPAIR_MAX_SLEW_PCT = 30 keeps 30 % slew margin in that repair (default 10 %), enough to cover the slow corner. The second series repeats the procedure from 3.5 ns with this configuration.

5.1 LibreLane Runs

		Series 1: required keys		Series 2: final configuration				
Run	Period	Setup	Slew	Setup	Hold	Typical	Slew	Result
1	3.5	−2.029	11	−2.032	−0.227	+0.586	0	setup, hold
2	4.0	−1.529	11	−1.532	−0.127	+1.086	0	setup, hold
3	4.5	−1.028	11	−1.028	−0.027	+1.588	0	setup, hold
4	5.0	−0.528	11	−0.528	+0.073	+2.088	0	setup
5	5.5	−0.146	11	−0.146	+0.129	+2.542	0	setup
6	6.0	−0.151	11	−0.151	+0.130	+2.887	0	setup
7	**6.5**	+0.041	11	**+0.041**	+0.129	+3.260	**0**	**passed**

Table 4: All LibreLane runs: worst-corner setup and hold slack (ns), typical-corner setup slack (ns) and maximum-slew violations. Every timing failure is at the slow ss_100C_1v60 corners; DRC (Magic and KLayout) and LVS are 0 in all 14 runs

5.2 Final Results

The post-route maximum frequency follows from the final period and setup slack:

$$F_{\text{max,post-route}} = \frac{1000}{\text{CLOCK_PERIOD} - \text{slack}} = \frac{1000}{6.5 - 0.0413} = 154.83 \text{ MHz}. \tag{7}$$

This is the slack-derived estimate the assignment asks for; the clock actually configured, 6.5 ns, corresponds to 153.85 MHz.

Metric	Value
`CLOCK_PERIOD` used	6.5 ns
Number of runs needed	7 per series (3.5 to 6.5 ns in 0.5 ns steps), 2 series
Core area	17,759.5 µm^2 (die 22,500 µm^2)
Cells after synthesis (Part B)	63
Cells after place and route	1,893
Kinds of cells added	tap, decap, fill, timing-repair and clock-tree buffers
Wirelength	2,333 µm
Setup slack (hold slack)	+0.041 ns (+0.129 ns)
Post-route F_{max}	154.83 MHz
F_{max} from Part C	386.10 MHz
DRC errors	0 (Magic 0, KLayout 0, detailed router 0)
LVS errors	0 (Netgen: circuits match uniquely)
Slew / capacitance / fanout violations	0 / 0 / 0

Table 5: Physical design results of the final run (`pnr/metrics.json`)

Kind of cell	Library cells	Count
Logic and flip-flops	from LibreLane's own synthesis	68
Decoupling capacitors	`decap_3`, `decap_4`, `decap_6`, `decap_8`, `decap_12`	1,302
Fill	`fill_1`, `fill_2`	256
Well taps	`tapvpwrvgnd_1`	245
Timing-repair buffers	`buf_2` (18), `clkdlybuf4s50_1` (1)	19
Clock-tree buffers	`clkbuf_16`	3
Total		**1,893**

Table 6: Cells in the final layout, counted from the routed netlist. LibreLane runs its own synthesis, which gives 68 logic cells and flip-flops instead of the 63 of Part B

From these results we observed that:

- The post-route F_{max} is 2.5 times lower than in Part C. At the same typical corner as Part C the routed design would allow $1000/(6.5 - 3.2603) = 308.7$ MHz; the rest of the reduction comes from slow-corner signoff (Section 6.4).
- The 1,830 additional instances are 1,803 physical-only cells without a logic function (the rows are only 7 % utilised and every empty site is filled), 22 buffers (19 for timing repair, 3 for the clock tree), and 5 extra logic cells because LibreLane's own synthesis maps the RTL slightly differently ($1803 + 22 + 5 = 1830$).
- The post-routing repair replaced the reset buffer with a stronger `buf_2`. This removed all 11 slew violations without changing the setup slack, the cell count or the critical path. The final

layout has no slew, capacitance or fanout violations, and reset recovery and removal pass at the slow corner (+4.39 ns and +0.37 ns).

5.3 Layout



(a) Whole 150 µm × 150 µm die: cell outlines, met1 rails, met4/met5 power straps, met2 (red) and met3 (green) routing, and I/O pins on the edges

(b) 24 µm × 24 µm detail at the centre of the logic: cell rows between met1 power rails, local interconnect, met1 wires and met2 risers with vias

Figure 4: Final layout `pnr/mult_array.gds` in KLayout 0.30.12 (subset of the SKY130 layers)

6 Answers to the Questions

6.1 Logic Cells after Synthesis

Synthesis produced 63 cells, of which 16 are flip-flops, so **47 cells** implement the logic rather than $16 + 12 = 28$. The RTL gates are not mapped one to one: after flattening, ABC optimises the whole logic network and selects whichever SKY130 cells are cheapest. A full adder has two outputs, and ABC does not use the two-output SKY130 `fa` cell, so each adder needs at least a carry cell (`maj3`, or an `a21oi`/`nand2` combination) and a sum cell (`xor3`/`xnor3`, or two two-input XOR/XNOR cells). Conversely, adders with constant inputs ($c_i[0] = 0$ and $r_0[3] = 0$) reduce to half adders or single gates, and many AND gates are absorbed into complex cells such as `a21oi`, `o311ai`, `o2111ai`, `nand3` and `nand4`. Only three `maj3` and three three-input XOR/XNOR cells remain as complete full-adder functions.

6.2 Critical Path

The critical path starts at the input register `a_reg[1]` (flip-flop `_087_`) and ends at the output register bit `p[6]` (`_100_`). It contains 10 cells, the two flip-flops and eight logic cells, with a data arrival time of 2.457 ns (Table 3). Traced back to the RTL it crosses FA(1,0), FA(1,1), FA(2,0), FA(2,1), FA(2,2), FA(3,1), FA(3,2) and FA(3,3): **eight full-adder stages, matching the $2N = 8$ architectural estimate**. The cell count equals the stage count because six stages each map to a single complex cell, while the first two stages are merged with the AND gates into two cells. Because STA does not check sensitization, this structural path is a conservative bound on the real worst-case delay (Section 4).

6.3 Flip-flop Share of the Area

The 16 flip-flops occupy 400.384 of 740.710 µm^2, or **54.05 %** of the cell area. A `dfrtp_1`, a D flip-flop with asynchronous reset, is a large cell of 25.024 µm^2: 3.5 times the average logic cell (340.33/47 = 7.24 µm^2). The number of registers grows linearly with the operand width ($4N = 16$ bits for a, b and

the $2N$-bit product), whereas the array logic grows as N^2. At $N = 4$ the array is so small that the fixed register cost dominates; in a wider multiplier the logic would take over.

6.4 Post-route Maximum Frequency and Added Cells

Part C assumes ideal conditions: the typical corner, no wire parasitics, an ideal clock and the bare netlist. LibreLane signs off the real layout instead:

- at the slow corner `ss_100C_1v60` (1.60 V, 100 °C), where cells are about 2.4 times slower: the launch flip-flop's clock-to-output delay is 0.97 ns instead of 0.40 ns, and each `maj3` takes about 0.92 ns instead of 0.38 ns;
- with extracted wire resistance and capacitance;
- with a real clock tree: the clock reaches the launch flip-flop after 0.55 ns but the capture flip-flop after only 0.50 ns, so about 0.04 ns of skew works against setup (the common insertion delay itself cancels), plus a 0.25 ns clock uncertainty;
- with a longer setup time (0.29 ns).

Together these give **154.83 MHz instead of 386.10 MHz**; measured at the typical corner, the routed design would reach 308.7 MHz. The flow added four kinds of cells:

- **Well taps (245)** tie the n-well and substrate to VPWR and VGND at regular intervals to prevent latch-up.
- **Decoupling capacitors (1,302)** sit between VPWR and VGND, supply local current spikes and reduce supply noise.
- **Fill cells (256)** close the remaining gaps so that wells, implants and power rails are continuous and density rules are met.
- **Buffers (22):** 19 repair buffers fix slew, capacitance and fanout on long or heavily loaded nets such as the reset and input nets, and three `clkbuf_16` cells form the clock tree that delivers a sharp clock edge to all 16 flip-flops with little skew.

6.5 DRC and LVS

DRC (Magic and KLayout) checks that the layout geometry obeys the foundry rules on every layer: minimum width and spacing, via and contact enclosure, minimum area, density and antenna rules. LVS (Netgen) extracts the devices and connections from the layout and checks that they form exactly the circuit of the netlist, with the same cells, nets and pins and no shorts or opens. Both must pass before manufacture. A DRC error means the masks cannot be printed reliably, which causes shorts, opens or low yield, and the foundry will reject the design. An LVS mismatch means the fabricated chip would be a different circuit from the one that was simulated and timed, which invalidates all of the verification. Our final layout has no DRC and no LVS errors.

6.6 Scaling to Eight Bits

Using the formulas of Section 2 of the assignment with the delay per stage measured here ($2.457 - 0.398 = 2.059$ ns for eight stages, about 0.257 ns per stage):

$$T_{\min}(N) \approx t_{\text{clk-q}} + 2N \cdot t_{\text{stage}} + t_{\text{setup}} = 0.398 + 2N \times 0.257 + 0.134 \text{ ns}. \tag{8}$$

Quantity	$N = 4$	$N = 8$	Ratio
AND gates, N^2	16	64	4.0
Full adders, $N(N-1)$	12	56	4.7
Flip-flops, $4N$	16	32	2.0
Critical path, $\approx 2N$ full-adder delays	8	16	2.0
$T_{\min}$ (ns)	2.59	4.65	1.8
$F_{\max}$ at the typical corner (MHz)	386	215	0.56

Table 7: Estimated effect of doubling the operand width

The number of full adders grows quadratically, but the critical path only doubles because it grows linearly with N. The latency remains two cycles, but each cycle becomes almost twice as long.

7 Observations and Conclusions

7.1 Conclusions

Summary

We designed, verified and implemented a registered 4-bit array multiplier from RTL to a DRC-, LVS- and electrically clean GDSII layout on SKY130. The final design closes at a 6.5 ns clock (153.85 MHz) with +0.041 ns setup slack, giving a post-route $F_{\max}$ estimate of 154.83 MHz, in a 150 µm × 150 µm die with 1,893 cells and 2,333 µm of wire, and no slew, capacitance or fanout violations.

7.2 Team Contributions

Team Member Contributions

Manas Inamdar (2023102052):

- Set up the repository and the IIC-OSIC-TOOLS environment
- Wrote the RTL (`full_adder.v`, `mult_array.v`) and the exhaustive testbench (`tb_mult.v`)
- Wrote `synth.tcl`; ran synthesis and the gate-level simulation (Part B)
- Wrote `sta.tcl`; found $T_{\min}$, $F_{\max}$ and the critical path (Part C)
- Wrote `config.json` and carried out the LibreLane runs to timing closure (Part D)

Soham Jahagirdar (2023102046):

- Set up the pinned tool image and reproduced the complete flow, Parts A–D
- Fixed the reset-net slew violations with post-routing repair and re-ran the full LibreLane sequence
- Strengthened the testbench and verified the routed netlist in simulation
- Traced the critical-path cells back to the full adders of the array
- Prepared the README, the figures and this report

Joint: design review, analysis of the results, answers to Questions 1–6, and the final review of the code, results and report.

Declaration on the use of AI tools

We used an AI assistant (Claude, Anthropic) to cross-check our flow. It re-ran the simulations, synthesis, static timing analysis and LibreLane runs, mapped the critical-path cells to the RTL, produced the waveform figure, the block diagram and the layout renders, and helped draft the README and this report. Every number in this report comes from the open-source tools listed in Section 1. Both team members have reviewed every file, result and answers.